CooL: A Verifiable Evidence System for Consequential Actions

Northwind Cipher
github.com/Northwind-Cipher/cool-sdk

November 2026 · describes the published specification `cool-nwc` 4.0.0

Abstract. When software acts on someone's behalf (an AI agent issuing a refund, a pipeline deploying a release, a service granting access), the record of what happened is usually kept by the same party whose conduct is in question. Logs can be edited, omitted or rewritten, and an auditor who wants certainty must trust the operator. We propose a system in which every consequential action leaves evidence that anyone can verify offline, without an account and without trusting the operator. Events are encoded canonically, committed by domain-separated hashes and signed with a hybrid post-quantum signature. They are appended to a Merkle transparency log whose tree heads are co-signed by independent witnesses and anchored to external time sources. Before an action runs, an authorization protocol binds who requested it, under whose delegated authority, with which parameters and under which policy version. The policy decision can be re-executed by any verifier. An enforcement gateway that owns the tool refuses anything it cannot verify, and signs what actually happened. Where execution happens inside a trusted execution environment, the signing key is bound to hardware attestation. Verification never reports a gap as success: every result is one of *verified*, *invalid*, *unsupported*, *stale*, *incomplete* or *indeterminate*.

1. Introduction

Organisations increasingly delegate consequential decisions to software. An agent that reads a support ticket and issues a refund, a build system that ships a binary to production, an access broker that grants a credential: each is an action with financial, legal or safety consequences. When something goes wrong, the questions are always the same. Who did this? Were they allowed to? Under which rules? What exactly happened? Is the record complete?

Today those questions are answered from logs, and logs have a structural weakness: they are written, stored and served by the operator. A log can show an auditor one history and an investigator another. It can drop the one entry that matters, or be edited after the fact. Even an honest operator cannot prove a negative, namely that nothing was removed. Signing log lines helps only a little, because the operator holds the keys. Timestamps help only a little, because the operator sets the clock. As a result, every assurance about software behaviour reduces to trust in an institution, and that trust is hardest to extend exactly when it is most needed: after an incident, in a dispute, or to a regulator.

What is needed is an evidence system based on verification rather than trust. It should let any party check a record independently, offline, against keys they choose to trust. It must make omission and equivocation detectable, bind an action to the authority under which it was taken, and report uncertainty honestly instead of collapsing it into success. In this paper we describe such a system. It does not prove that software made a good decision; nothing can. It proves who decided, under what authority and policy, what was actually executed, and that the record has not been altered, forked or quietly truncated.

2. Evidence Envelopes

We define a unit of evidence as an *envelope*. Its core is a typed record: an identifier, an event type and payload schema, the time the event occurred and the time it was observed, the source system, the actor, the action, the resource, references to earlier evidence, and an optional sequence number and attestation reference. Payloads are typed. Fifty event families are defined, including authorization requests and decisions, tool invocations, external confirmations, deployments, credential use and AI inference. Validation is strict, and fields that look like secrets are rejected unless they carry a digest. Parameters are carried as commitments, never in plaintext (Section 10).

The core is serialized with the core deterministic encoding of CBOR [3], so that one record has exactly one byte representation. The envelope digest is a SHA-256 hash over a domain label and those bytes, and the signature covers a second, distinct label and the digest:

$$d = \mathit{SHA\text{-}256}(\texttt{"cool.envelope.v1\0"} \,\|\, \mathit{CBOR}_{CDE}(\mathit{core})) \qquad m = \texttt{"cool.envelope.sig.v1\0"} \,\|\, d$$

$$\sigma = (\ \mathit{ML\text{-}DSA\text{-}65.Sign}(sk_1, m),\ \mathit{Ed25519.Sign}(sk_2, m)\ )$$

The signature is hybrid. It contains an ML-DSA-65 signature [4] and an Ed25519 signature [5] over the same message, and a verifier accepts it only if both verify. Forging an envelope therefore requires breaking both schemes. The classical scheme protects against an undiscovered flaw in the new lattice scheme, and the lattice scheme protects against a future quantum adversary. The domain labels ensure that a signature over one kind of object can never be replayed as a signature over another. Every signed object in the system has its own label: decisions, delegations, confirmations, principal records and tree heads.

Envelopes refer to each other by digest. A decision references the request it answers, and a confirmation references the decision it executed. Causality is therefore established by hashes rather than by timestamps, which any party could set.

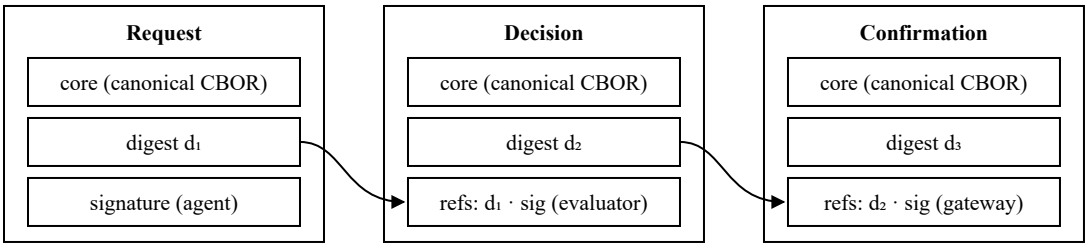


Figure 1. Envelopes linked by digest.

3. Trust Snapshots

A valid signature says only that some key signed some bytes. Whether that key should be believed is a separate question, and we keep the two apart. A verifier supplies a *trust snapshot*: a dated list of keys, each with an owner, the purposes it is accepted for (`evidence`, `log`, `witness`, `evaluator`, `enforcement`, `identity_issuer` and others), a validity window and a status. A key trusted only to sign tree heads cannot sign a decision. Nothing is fetched at verification time: the snapshot is an explicit input, so two verifiers with the same evidence and the same snapshot always reach the same result.

Keys change over time. A *rotated* key is refused for signatures dated after its successor took over. A *revoked* or *compromised* key raises a subtler problem. Its holder can produce a signature with any claimed date, so a claimed date proves nothing. We accept such a signature only with *independent* proof that it existed before the revocation: inclusion in a log checkpoint, or an external time anchor (Section 5). A

snapshot may declare a maximum age. When it is older than that, revocations since then are unknown, and the result is *stale* rather than *verified*.

4. Transparency Log

Signatures prevent alteration, but not omission or equivocation. An operator could sign two different histories and show each to a different party, or simply never show an inconvenient record. To make history append-only and globally consistent, every envelope digest is appended as a leaf of a Merkle tree, following Certificate Transparency [6]:

$$leaf = SHA\text{-}256(\texttt{0x00} \parallel d) \qquad node = SHA\text{-}256(\texttt{0x01} \parallel left \parallel right)$$

The log periodically signs a *tree head*: its identifier, the tree size, the root hash and a timestamp. With the tree head comes an *inclusion proof* for each envelope, which is the logarithmic-size path of sibling hashes from the leaf to the root. Given two tree heads, a *consistency proof* shows that the larger tree extends the smaller one without changing any earlier leaf. A log that rewrites history, or forks it, therefore produces two tree heads that cannot be proven consistent, and anyone holding both has cryptographic proof of misbehaviour.

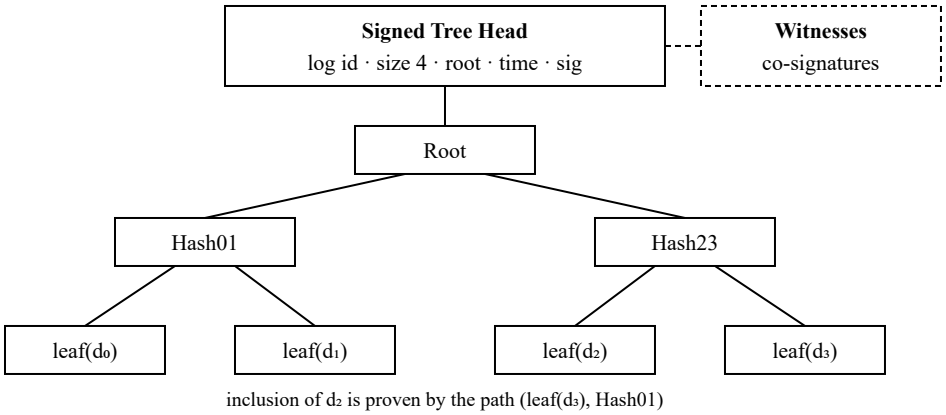


Figure 2. Envelope digests as leaves of a Merkle tree; the signed tree head is co-signed by witnesses.

5. Witnesses and Time

A consistency proof convicts a forking log only if someone compares the tree heads. *Witnesses* do that comparing. A witness is an independent party that receives each new tree head, checks that it is consistent with the last head it accepted, and only then adds its own co-signature. A witness refuses to co-sign a head that does not extend its history. A verifier can demand co-signatures from a threshold of witnesses it trusts. To show different histories to different parties, a log must then corrupt that many witnesses or partition them permanently. The guarantee depends on witnesses actually being operated by different organisations. That is an operational property the cryptography cannot supply, and we state it as such.

Timestamps inside an envelope are assertions made by its source. Where trusted time matters, for revocation (Section 3) or for disputes, a tree head or an envelope digest is anchored externally. Two independent mechanisms are supported. The first is an RFC 3161 time-stamp token [7]: a certificate authority dedicated to time signs that a digest existed at a given instant, and the token is verified offline against a pinned root. The second is an OpenTimestamps proof [8], which commits the digest into a Bitcoin block [1]. Either one turns "the operator says this existed then" into "a third party proves this existed by then". Neither says anything about content, which is the job of the signature.

6. Authorization

Evidence of an action is most useful when it also shows the authority behind it. We define five signed protocol objects. Each is domain-separated and verified against a purpose in the trust snapshot.

1. **Principal.** An identity issuer asserts that a principal exists, a human, workload or agent, and which keys it holds. Status changes and key rotations are signed events, so a verifier can reconstruct the principal's state *at the time of the action*, not merely today.
2. **Delegation.** A principal grants another a scope: actions, resource prefixes and an amount ceiling, for an audience and a validity window. Delegations chain. A verifier checks that each link's scope is a subset of its parent's, so privilege can never be amplified. It also checks the chain depth, cycles, revocation and that the chain ends at a root the verifier accepts.
3. **Request.** The agent states precisely what it wants to do: action, resource, audience, policy selector, a fresh nonce and an expiry. Its parameters are committed rather than disclosed (Section 10).
4. **Decision.** An evaluator applies a policy and signs the result, which is *permit*, *deny*, *indeterminate* or *error*. The decision is bound to the digest of the request, the digest of the exact policy version and a digest of the facts it used.
5. **Confirmation.** The system that owns the action signs what it did: completed, rejected or failed, with a digest of the response (Section 7).

A decision can be believed in two ways. A *trusted* decision asks the verifier to trust the evaluator's key. A *reproducible* decision asks for no trust at all. The policy language is deterministic and its evaluator is public, so a verifier given the policy and the facts re-executes the evaluation and checks that it reaches the same result, under the same policy digest. A missing fact is treated as unknown, never as false. When the policy changes later, the old decision still verifies against the version it actually used. Presenting the new version as the old decision's policy fails on the policy digest.

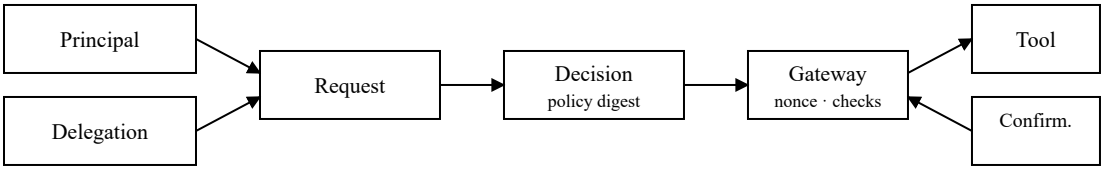


Figure 3. From identity and delegated authority to a confirmed action.

7. Enforcement

Evidence that an action was not permitted is of little comfort if the action happened anyway. The decisive control is therefore placed in an *enforcement gateway* that sits with the tool, in front of the system that owns the action, rather than in the agent. Before executing, the gateway checks:

- that the requested action and resource match the tool, the audience is its own, and the presented parameters open the request's commitment;
- the decision's signature and the evaluator's trust, the binding to this request, the policy (by replaying it when the policy and facts are supplied), the decision's freshness, and the requester's state at event time;
- that the delegation chain authorises this principal for this action, resource and amount;
- that the workload now executing is the one the request was authorised for (the artifact and configuration digests);

- and finally that the request's nonce is unused, consuming it atomically.

Any failure, and any check whose answer is unknown, produces a refusal. A replay store that cannot answer is treated as a refusal, never a permission. On a single host the nonce is consumed with an exclusive file create. Across machines it is consumed with an atomic conditional write in a shared store. Whatever the outcome, the gateway signs a *confirmation* that references the request and decision and states whether the action completed, was rejected (with the reason) or failed. The tool's owner, not the agent, attests to what happened. Because the policy check is repeated at the boundary, a misconfigured evaluator that permits too much is still stopped by the delegation ceiling.

8. Hardware Attestation

Where evidence is produced inside a trusted execution environment, the sealing key can be bound to the hardware's statement about what code is running. The sealer generates a fresh hybrid key inside the enclave and asks the platform to attest to it. On AWS Nitro Enclaves the attestation's `user_data` carries

$$u = SHA\text{-}256(\texttt{"cool.nitro.key-binding.v1\0"} \,\|\, CBOR(id,\ pk_{ML\text{-}DSA},\ pk_{Ed25519}))$$

and on Intel TDX and AMD SEV-SNP the equivalent field is `report_data`. A verifier checks three things. The attestation chains to the vendor root, which is pinned in code and re-checked when the module loads. The measurements match a pinned policy, and debug mode is refused. The bound key is exactly the key that signed the evidence. A valid result supports one claim: *code with these measurements held this key*. It does not show that the code is free of defects, and it does not stop the host from denying service.

Each platform is supported through an adapter with one contract. Every adapter returns the platform's own claims unaltered, with an honest outcome. For SEV-SNP the verifier checks the chain from VCEK or VLEK to the pinned AMD root, then checks that the certificate's TCB equals the reported TCB and that its hardware identifier equals the chip identifier. That way a certificate for one chip or firmware level cannot vouch for another. For GPUs, the NVIDIA attestation service's signed appraisal is verified offline and can be bound to a CPU report through a shared nonce.

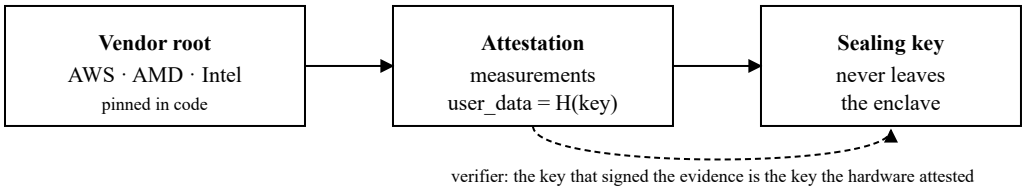


Figure 4. Binding the evidence key to hardware attestation.

9. Verification and Honest Outcomes

Verification of an envelope is a set of separate checks, each reporting *pass*, *fail*, *unknown*, *not applicable* or *not checked*, with a reason. The eleven envelope checks are: schema, canonical encoding, digest, signature, signer trusted, signer authorised, timestamp, inclusion proof, checkpoint trusted, attestation and evidence completeness. A bundle adds the manifest, the exporter's signature, attachment binding and sequence completeness. Decisions, attestations and time-stamp tokens have checks of their own.

Collapsing these into a single bit would destroy information. A missing witness is not a forgery, and an unknown platform is not a valid one. We therefore summarise every report with one of six outcomes, chosen by a fixed precedence:

$$invalid > unsupported > stale > incomplete > indeterminate > verified$$

Outcome	Meaning	Typical cause
invalid	a check failed	altered bytes, untrusted signer, wrong policy digest
unsupported	the verifier cannot judge this	unknown algorithm, schema or platform
stale	true once, not shown to be true now	expired certificate or attestation, old trust snapshot
incomplete	evidence is missing	a gap in sequence numbers, a missing confirmation
indeterminate	a required input was not supplied	no measurement policy, no nonce, no trust anchor
verified	every applicable check passed	—

Completeness deserves a word. Evidence from one source can carry an epoch and a sequence number. A verifier that sees sequence numbers 7.0, 7.1 and 7.3 reports *incomplete*, naming 7.2, rather than presenting the history as clean. The last event of a sequence can only be shown missing once a later one arrives. A source that emits sequenced heartbeats bounds that window (Section 11).

Evidence travels in *bundles*: envelopes, inclusion proofs, trust snapshots and the signed protocol objects they reference, under an exporter's signature. A bundle verifies with the public verifier, offline, by a party with no account and no network access. This also means a customer who stops using any service keeps evidence that remains verifiable indefinitely.

10. Privacy

Evidence must be shareable with auditors without disclosing the data it describes. Request parameters and context are therefore represented only by commitments:

$$c = \mathit{SHA\text{-}256}(\texttt{"cool.authz.params.v1\0"} \,\|\, \mathit{salt} \,\|\, \mathit{CBOR(NFC(params))})$$

$$\text{or, keyed:}\quad c = \mathit{HMAC\text{-}SHA\text{-}256}_k(\texttt{"cool.authz.params.v1\0"} \,\|\, \mathit{salt} \,\|\, \mathit{CBOR(NFC(params))})$$

The salt is 128 random bits, held by the requester. Values are normalised (Unicode NFC, safe integers only, no floating point), so that equal values always commit equally. The holder can later reveal the salt and value, and anyone can check them against the commitment. For low-entropy values such as amounts or account numbers, a keyed commitment under a tenant key resists guessing even if a salt leaks. Selective disclosure follows the same pattern for individual envelope fields.

The log never needs the content. It stores digests. Envelope bodies can therefore be deleted under a retention policy while the leaf stays in the tree, and history remains provable. Each deletion is itself a signed envelope naming the digest and the policy, and a later export reports the deleted body as missing, so a verifier sees *incomplete* rather than a clean history. Destroying a commitment's salt makes it impossible ever to open, while the evidence stays structurally valid. A legal hold suspends deletion for a time range.

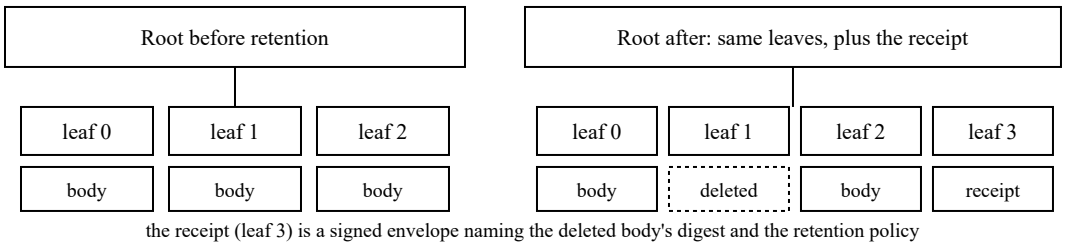


Figure 5. Retention deletes bodies, never leaves: the tree still proves the history, and a signed receipt records the deletion.

11. Calculations

Forgery.

To forge an envelope, an adversary must produce both a valid ML-DSA-65 signature and a valid Ed25519 signature on a message of its choosing, under keys a verifier trusts. Against a classical adversary the work is at least that of breaking Ed25519, about 2^{128} operations. Against an adversary with a large quantum computer, Ed25519 falls, but ML-DSA-65 (NIST security category 3) still stands. The hybrid is never weaker than its stronger component.

Equivocation.

Suppose a log shows a victim a forked history. The fork is discovered if any party compares the victim's tree head with the head the rest of the world sees. If each of z independent relying parties performs that comparison against an honest witness with probability q, the probability that the fork survives all of them is

$$P = (1 - q)^z$$

```
double ForkSurvives(double q, int z)
{
    double p = 1.0;
    for (int i = 0; i < z; i++)
        p *= (1.0 - q);
    return p;
}
```

A verifier that demands co-signatures from t witnesses forces $q = 1$ for itself, unless the adversary also controls those t witnesses. Even occasional comparisons by others drive the survival probability down exponentially:

z	q = 0.05	q = 0.10	q = 0.25	q = 0.50
10	0.5987	0.3487	0.0563	0.0009766
50	0.0769	0.0052	5.7×10^{-7}	8.9×10^{-16}
100	0.0059	2.7×10^{-5}	3.2×10^{-13}	7.9×10^{-31}
500	7.3×10^{-12}	1.3×10^{-23}	3.4×10^{-63}	3.1×10^{-151}

Guessing committed parameters.

A salted commitment hides its value with 128 bits of salt, so an adversary who does not hold the salt must search 2^{128} possibilities, whatever the value. If the salt is disclosed, for example to an auditor, a value drawn from a small space can be enumerated: a refund amount of up to 10^6 cents falls in a million guesses. The keyed form prevents this. Without the tenant key, no guess can be tested at all, whatever the size of the space.

Undetected omission.

Dropping a sequenced event that is followed by another is detected with certainty by any verifier that sees the later event. Only the last event before an observation can disappear unnoticed. If a source emits a sequenced heartbeat every h seconds, an omission can stay hidden for at most h seconds plus the delivery delay. After that, the gap appears in every bundle that contains the following heartbeat.

Replay.

Each request carries a fresh nonce and an expiry, and the gateway consumes the nonce atomically before executing. Of any number of concurrent submissions of one authorization, at most one executes. This is tested with 25 parallel submissions on one host, and with competing clients against a shared store. The nonce need only be remembered until the request expires, after which the expiry refuses it on its own.

12. Status and Limitations

A system that asks to be trusted for honesty about evidence must be honest about itself. Every capability of the reference implementation carries exactly one label, defined in code and enforced by its test suite. IMPLEMENTED means validated against genuine evidence or a real service. EXPERIMENTAL means built and tested, but not yet validated on the real hardware or cloud service. As of this specification:

Capability	Label	Basis
Envelopes, bundles, outcomes, transparency log, witnesses	implemented	adversarial and fuzz tests
Authorization protocol, reproducible decisions, gateway	implemented	adversarial and race tests; scenario demo
Replay protection, one host and Redis across machines	implemented	race tests; continuous test against a real Redis server
RFC 3161 and OpenTimestamps anchoring	implemented	a genuine DigiCert token, exhaustively bit-flipped
AWS Nitro attestation verification	implemented	a genuine AWS attestation document
AMD SEV-SNP attestation verification	implemented	a genuine Milan report; AMD roots pinned
Intel TDX via Phala dstack	implemented	recorded hardware validation
Sealer running in a Nitro Enclave on AWS hardware	experimental	deployment kit built; not yet run on AWS
NVIDIA confidential-computing results	experimental	real service keys; no real GPU result yet
AWS KMS, Google Cloud KMS, PKCS#11 signers	experimental	provider emulators; not yet run live
Independent third-party security review	planned	internal review done; review package prepared

Some limits are inherent to the approach and will not change:

- CooL proves that a record is authentic, consistent and complete within its sequence. It does not prove that the recorded statement is true, or that a decision was wise.
- Actions that bypass the gateway are not covered.
- An identity issuer or evaluator that the verifier trusts can lie within its authority. A reproducible decision removes the evaluator from that list, but the facts it consumed are only as good as their source.
- Attestation shows a measured launch state, not correct behaviour.
- Witness independence is operational.
- Revocation status is only as current as the trust snapshot.

Each of these limits is reported explicitly in the verification output.

13. Conclusion

We have proposed an evidence system for consequential actions that does not rely on trust in the operator. We started from envelopes: canonical records committed by domain-separated hashes and hybrid signatures. Signatures alone cannot reveal omission or equivocation, so we appended every digest to a Merkle log whose tree heads are co-signed by witnesses and anchored in external time. To connect evidence to authority, we added a protocol in which identity, delegated scope, a committed request and a policy decision are each signed by the party responsible. The decision can be reproduced by anyone holding the policy and facts. Enforcement is placed at the boundary that owns the action, which refuses whatever it cannot verify, consumes each authorization once, and signs what it actually did. Where hardware allows, the evidence key is bound to an attestation of the running code. Throughout, verification refuses to round uncertainty up: a missing record is *incomplete*, an old certificate is *stale*, an unknown platform is *unsupported*, and only a fully checked record is *verified*. The format, the verifiers and the authorization protocol are open, so that anyone can check the evidence without asking permission.

References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
2. S. Haber, W. S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.
3. C. Bormann, P. Hoffman, "Concise Binary Object Representation (CBOR)," RFC 8949, §4.2 Core Deterministic Encoding, 2020.
4. NIST, "Module-Lattice-Based Digital Signature Standard," FIPS 204, 2024.
5. S. Josefsson, I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, 2017.
6. B. Laurie, A. Langley, E. Kasper, "Certificate Transparency," RFC 6962, 2013.
7. C. Adams, P. Cain, D. Pinkas, R. Zuccherato, "Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)," RFC 3161, 2001.
8. P. Todd, "OpenTimestamps: scalable, trust-minimized, distributed timestamping with Bitcoin," 2016.
9. S. A. Crosby, D. S. Wallach, "Efficient data structures for tamper-evident logging," *USENIX Security Symposium*, 2009.
10. R. Merkle, "Protocols for public key cryptosystems," *IEEE Symposium on Security and Privacy*, 1980.
11. Amazon Web Services, "AWS Nitro Enclaves User Guide: Verifying the root of trust," 2026.
12. AMD, "SEV Secure Nested Paging Firmware ABI Specification," 2025.
13. Intel, "Intel Trust Domain Extensions (TDX) Module Architecture Specification," 2025.
14. Northwind Cipher, "cool-nwc 4.0.0: specification, schemas and test vectors," github.com/Northwind-Cipher/cool-sdk, 2026.